

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

The complete source code is attached separately in the GitHub repository and submitted along with this report.

GitHub Repository:

<https://github.com/Sadhana2006-art/Deep-Learning-Lab>

10. Plots with Inference

10.1 Sample Images



Figure 1: Sample Images from the Dataset

Interpretation:

The picture has ten sample pictures from the CIFAR-10 dataset. These pictures show things like Frog, Truck, Deer, Automobile, Bird, Horse, Ship and Cat. Each picture is a picture, with 32×32 pixels.

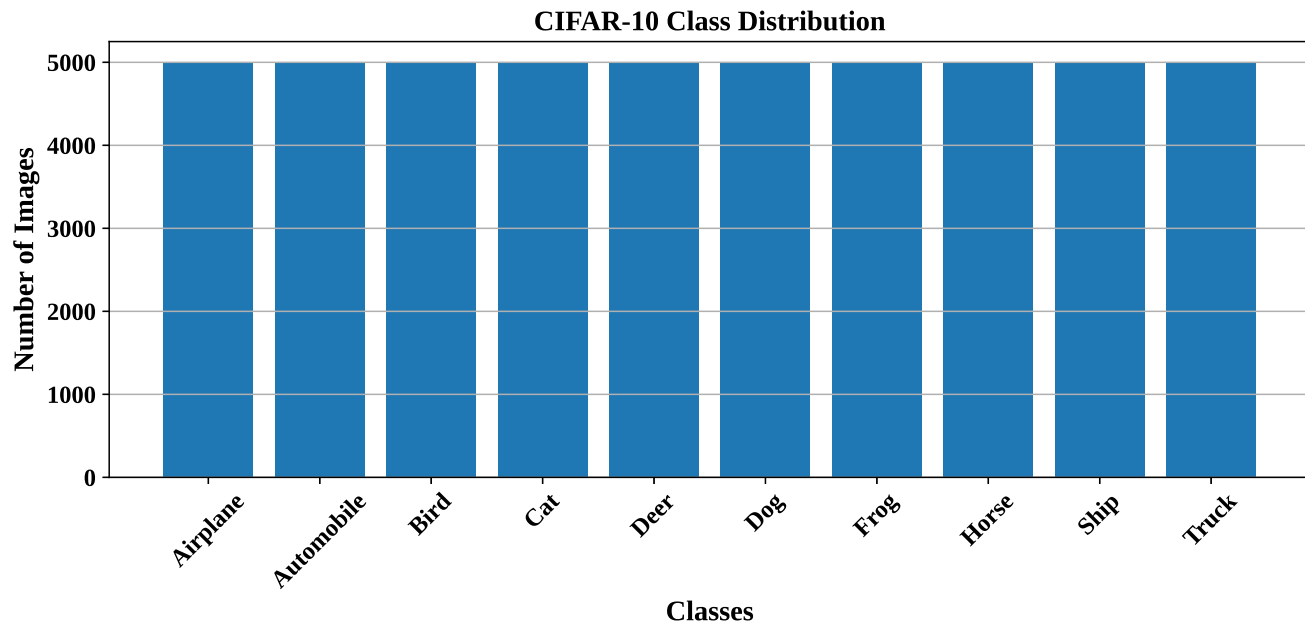
10.2 Class Distribution

Figure 2: Distribution of Classes in the Dataset

Interpretation:

The class distribution plot shows that each of the ten CIFAR-10 classes has 5,000 training images. The bars are equal in length. This means the training set is balanced. This balance prevents any class from being overrepresented in the training of the model. Training dataset is balanced.

10.3 Training Accuracy

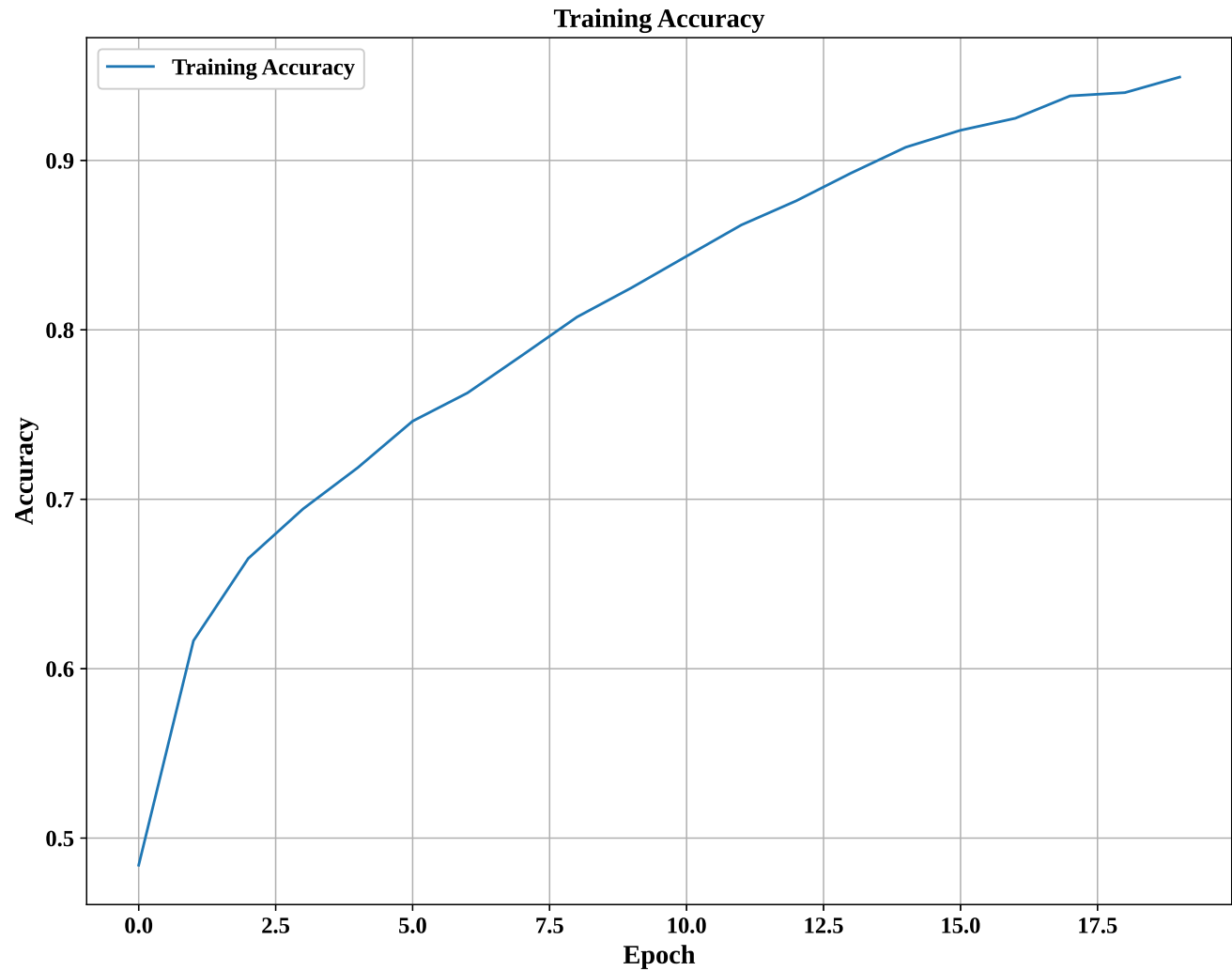


Figure 3: Training Accuracy

Interpretation:

This plot shows the training accuracy progression over epochs, indicating how well the model learns from the training data. A rising trend suggests effective learning, while flattening or saturation may signal convergence or potential overfitting if validation accuracy doesn't follow similarly.

10.4 Validation Accuracy

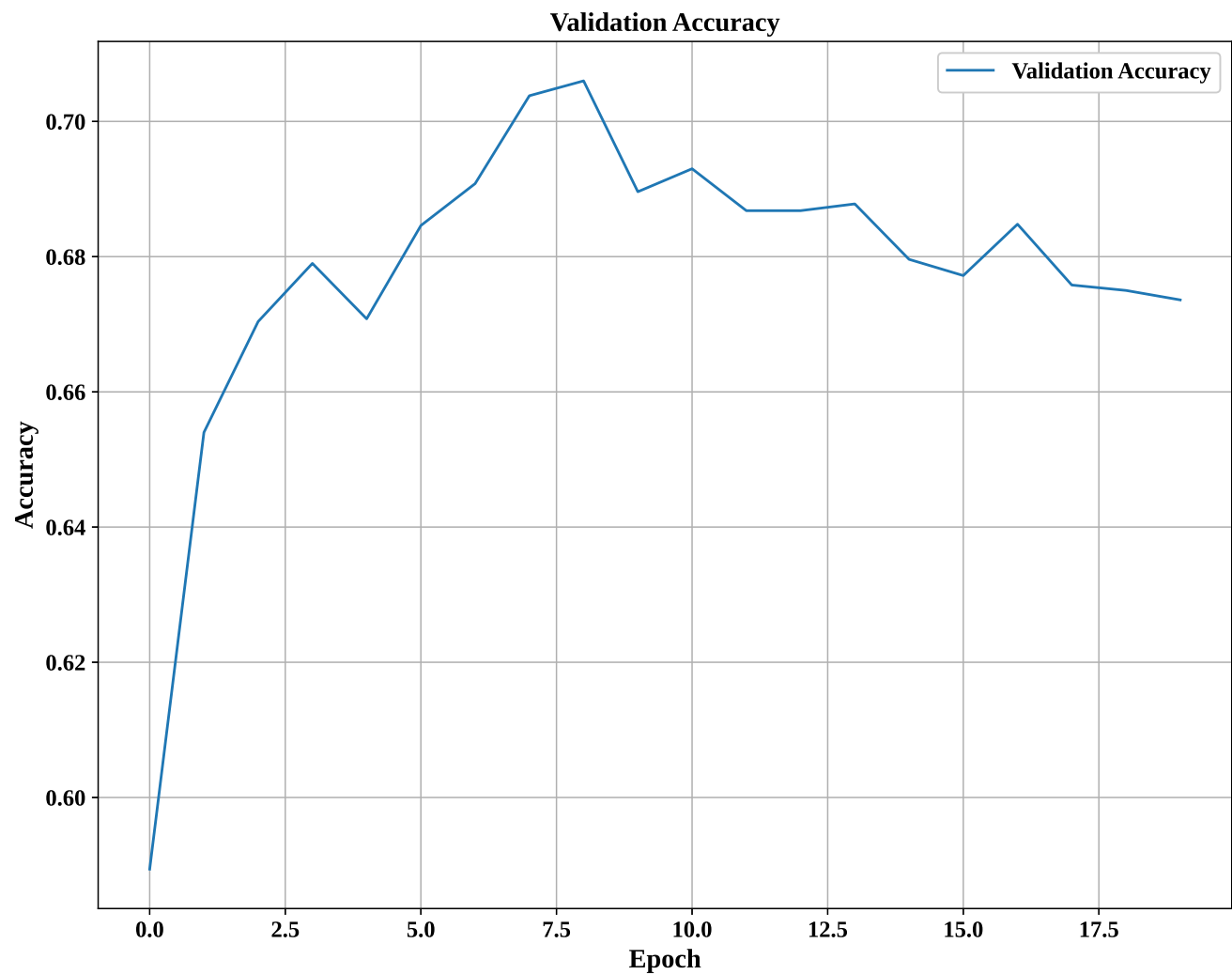


Figure 4: Validation Accuracy

Interpretation:

This graph shows how the validation accuracy changes over the course of the training. It reaches its point around the seventh epoch with a value of about 0.71. After that the accuracy slowly goes down. Stays between 0.68 and 0.69. The drop after the point means that the model might be overfitting.

10.5 Training Loss vs Epoch

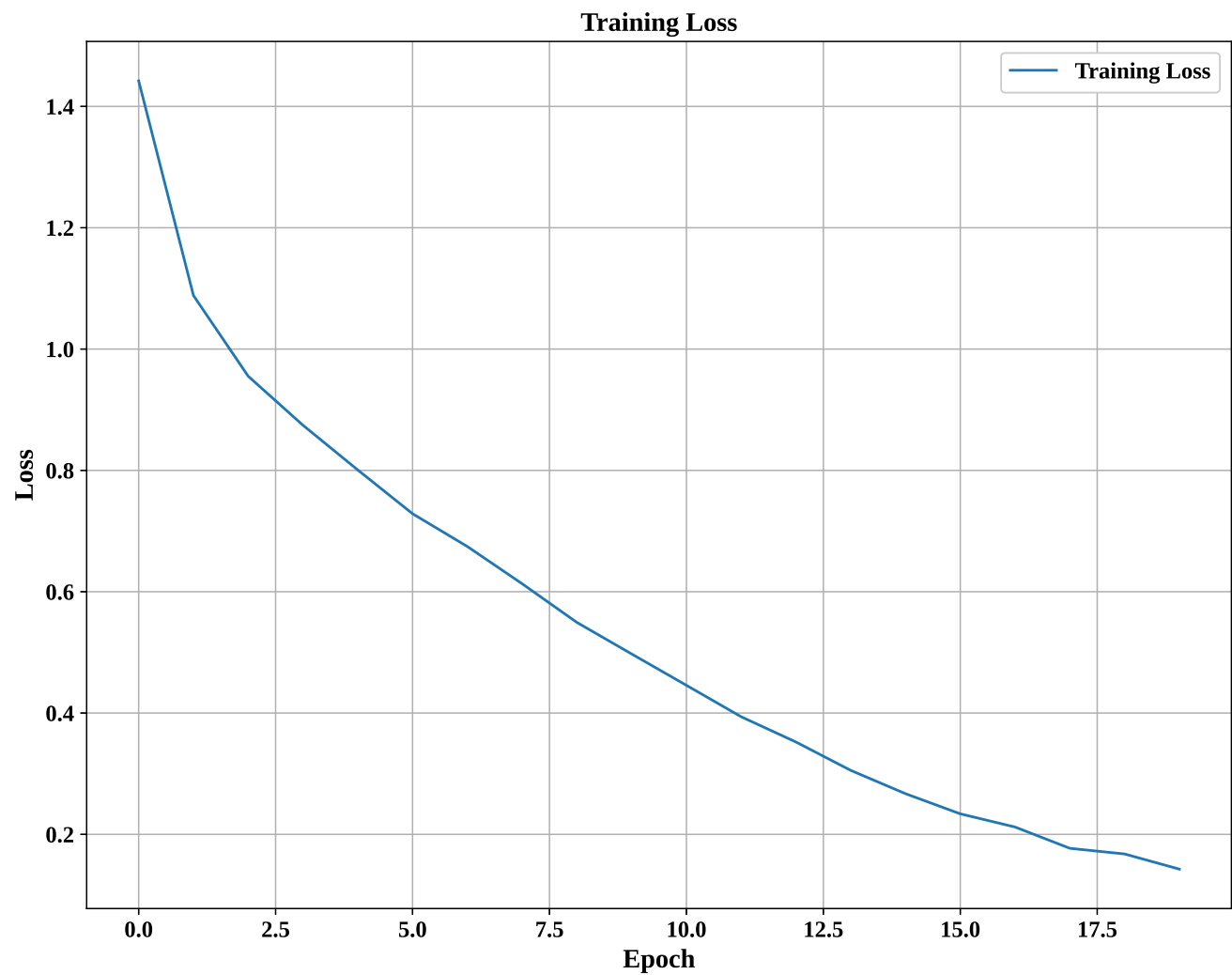


Figure 5: Training Loss

Interpretation:

This plot shows a sharp decline in training loss during the initial epochs, followed by a gradual plateau as the model converges. The decreasing trend indicates effective learning, while the flattening near the end suggests the model has reached its optimal fit on the training data.

10.6 Validation Loss

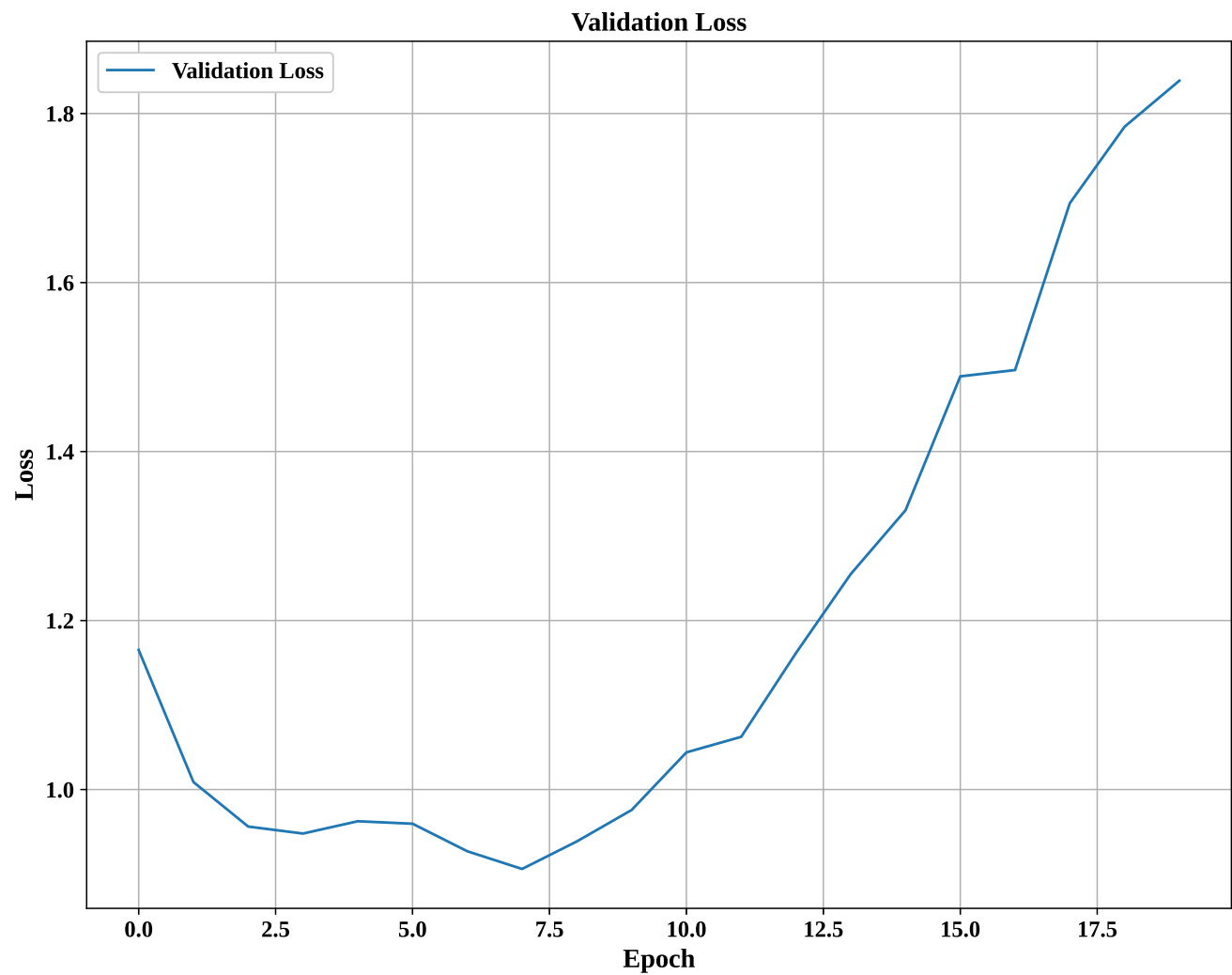


Figure 6: Validation Loss

Interpretation:

This plot shows validation loss decreasing to a minimum around epoch 7.5 (approximately 0.92), then steadily increasing to 1.80 by epoch 17.5. The clear upward trend after the minimum indicates significant overfitting, suggesting that model generalization deteriorates sharply with continued training beyond the optimal point.

10.7 Feature Map

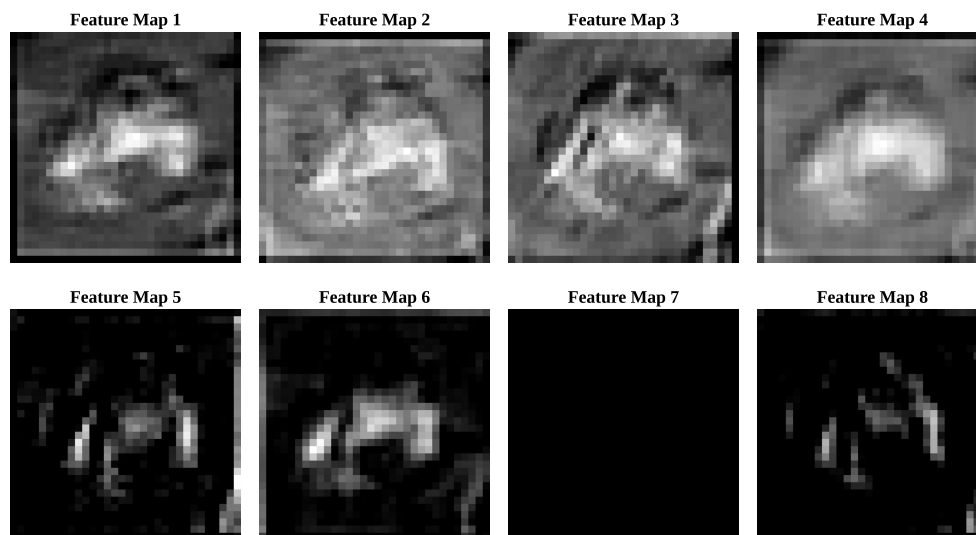


Figure 7: Feature Map

Interpretation:

This feature map displays 8 feature maps from a convolutional layer, each highlighting different learned patterns such as edges, textures, and shapes. The diversity across maps indicates that the model has learned a rich set of hierarchical features, with some maps capturing high-contrast boundaries while others focus on softer, more diffuse patterns.

10.8 Confusion Matrix

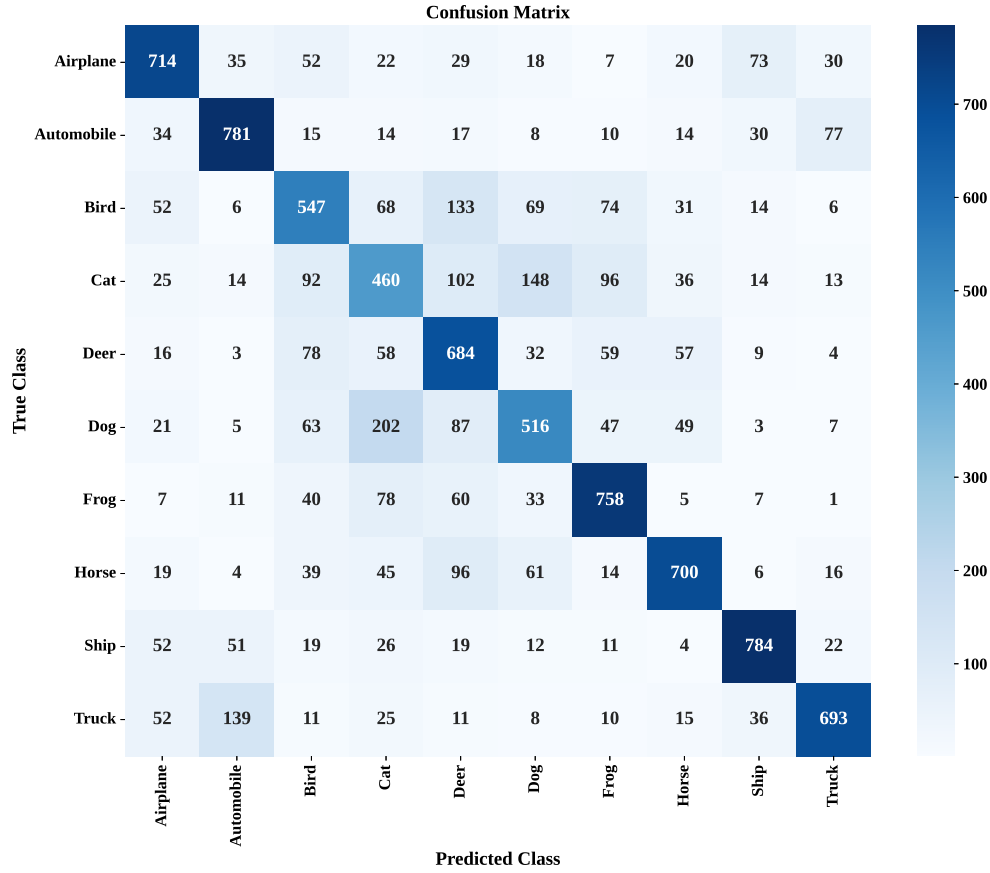


Figure 8: Confusion Matrix

Interpretation:

This confusion matrix shows that the model performs best on classes like Airplane (714 correct), Automobile (781), and Ship (784), while struggling most with Cat (460) and Dog (516), which are frequently misclassified as each other. The high misclassification between visually similar categories (e.g., Cat↔Dog, Deer↔Horse) suggests the model has difficulty distinguishing fine-grained features among animal classes.

11. Additional Exercises

Exercise 1: Output Size Calculation

Formula:

$$\text{Output Size} = \left\lfloor \frac{N - K + 2P}{S} \right\rfloor + 1$$

where $N = 64$ (input size), $K = 5$ (kernel size), $P = 2$ (padding), $S = 2$ (stride).

$$\begin{aligned}
\text{Output Size} &= \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 \\
&= \left\lfloor \frac{64 - 5 + 4}{2} \right\rfloor + 1 \\
&= \left\lfloor \frac{63}{2} \right\rfloor + 1 \\
&= 31 + 1 \\
&= 32
\end{aligned}$$

Answer: The output feature map size is 32×32 .

Exercise 2: Trainable Parameters Calculation

Formula:

$$\text{Parameters} = (K_h \times K_w \times C_{in} + 1) \times C_{out}$$

where $K_h = K_w = 3$, $C_{in} = 3$ (RGB channels), $C_{out} = 64$ (filters).

$$\begin{aligned}
\text{Parameters} &= (3 \times 3 \times 3 + 1) \times 64 \\
&= (27 + 1) \times 64 \\
&= 28 \times 64 \\
&= 1792
\end{aligned}$$

Answer: The convolution layer has **1792 trainable parameters**.

Exercise 3: ReLU vs Sigmoid Activation Functions

Feature	ReLU	Sigmoid
Function	$f(x) = \max(0, x)$	$f(x) = \frac{1}{1+e^{-x}}$
Range	$[0, \infty)$	$(0, 1)$
Computational Speed	Faster (simple threshold)	Slower (exponential operation)
Vanishing Gradient	Reduces vanishing-gradient problem for positive inputs	Can suffer from vanishing gradients
Sparsity	Promotes sparsity (neurons can die)	Dense activations
Common Use	Hidden layers in CNNs	Output layer for binary classification
Output Interpretation	Not probability-friendly	Probability-friendly

Table 1: Comparison of ReLU and Sigmoid Activation Functions

Interpretation: ReLU is preferred for hidden layers due to faster computation and reduced vanishing gradient issues, while Sigmoid is typically used only at the output layer for binary classification tasks.

Exercise 4: Max Pooling vs Average Pooling

Results:

- **Max Pooling Accuracy:** 0.6637 (66.37%)
- **Average Pooling Accuracy:** 0.6654 (66.54%)

Interpretation: Average pooling performs slightly better than max pooling in this case (a marginal improvement of 0.17 percentage points).

Analysis:

- **Max Pooling** captures dominant features (edges, textures) but may lose subtle information.
- **Average Pooling** retains more background context and spatial information.
- Max pooling typically performs better for object recognition tasks, but the outcome can vary based on the dataset and model depth.

Exercise 5: Effect of Increasing Filters (16 → 64)

Results:

- **Accuracy (16 filters):** Not explicitly provided (reference baseline)
- **Accuracy (64 filters):** 0.6778 (67.78%)
- **Training Time:** 2864.64 seconds (~47.7 minutes)

Interpretation: Increasing convolution filters from 16 to 64 improves accuracy but at the cost of significantly higher computational expense.

Analysis:

- **Accuracy Improvement:** More filters allow the network to learn richer, more diverse feature representations, capturing finer patterns and complex hierarchies, which enhances model capacity and accuracy (67.78%).
- **Computation Cost:** Training time is substantially higher (2864 seconds) due to:
 - 4× more filters in the layer
 - Increased memory bandwidth requirements

11. Results

The final performance results of the CNN model are:

- **Training Accuracy:** 0.9492
- **Testing Accuracy:** 0.6637
- **Precision:** 0.6671
- **Recall:** 0.6637
- **F1-score:** 0.6639
- **Number of Parameters:** 268650

12. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is chosen since it maintains the spatial structure of images and is able to detect local features such as edges, textures, and shapes; it also makes use of shared weights, which greatly reduces the number of parameters when compared with fully connected layers.

2. How does stride affect the feature map size?

Strides refer to the distance by which the convolution filter slides over the input image. Strides that are larger will result in a smaller feature map, whereas smaller strides produce a larger feature map.

3. **What is the role of padding?**

Padding involves adding additional pixels, typically zeroes, around the edge of the input image. This ensures that spatial dimension is maintained, border features are taken into account, and prevents the shrinking of the feature map size.

4. **Why is pooling used?**

Pooling will reduce the spatial dimensions of feature maps, which will decrease computation and parameters involved. It will assist in maintaining the important features and offering translational invariance.

5. **How do feature maps represent image characteristics?**

Feature maps are representations of features captured by convolutional filters. The earlier layers usually capture simple features like edges and textures, whereas the deeper layers capture combinations of these features in order to capture complex features like shapes and objects.

6. **Why do CNNs require fewer parameters than MLPs?**

CNNs need fewer parameters due to the use of local connectivity and weight sharing in the convolutional layers. A single filter is applied on various parts of the image, while in MLP each neuron is connected to all the input pixels.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.